

DESIGN AND ANALYSIS OF ALGORITHMS

LAB OBSERVATION

Question 1:

Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc", "car", "ada", "racecar", "cool"]

Output: "ada"

Example 2:

Input: words = ["notapalindrome", "racecar"]

Output: "racecar"

Aim:

To write a Python program to find the first palindromic string in a given array of strings.

Algorithm:

1. Read the number of words and the words from the user.
2. Check each word one by one.
3. Compare the word with its reverse.
4. If the word is equal to its reverse, store it as the result and stop.
5. If no palindrome is found, return an empty string.
6. Display the result.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
n = int(input("Enter number of words: "))
words = []

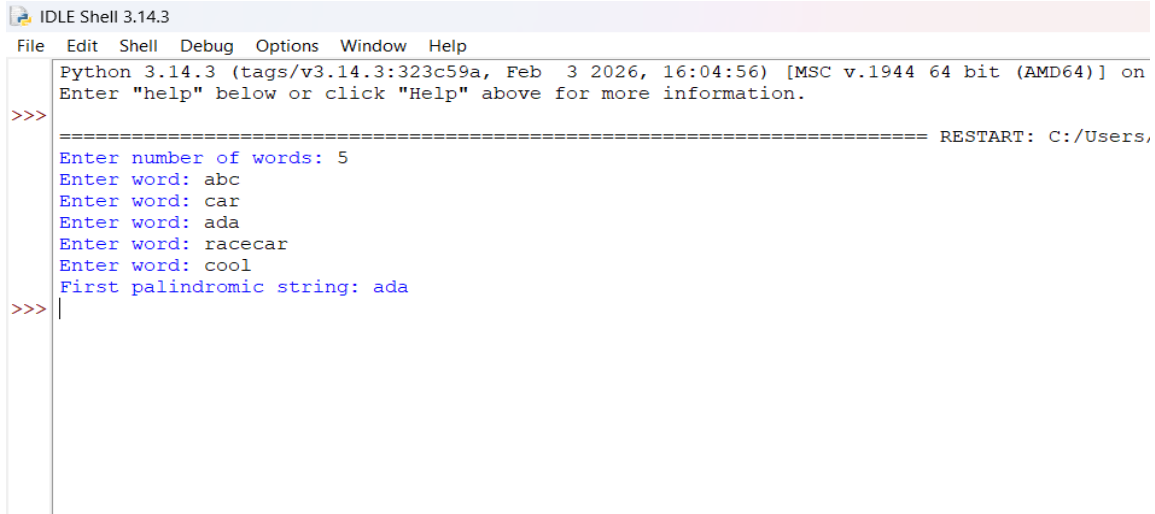
for i in range(n):
    words.append(input("Enter word: "))

result = ""

for word in words:
    if word == word[::-1]:
        result = word
        break

print("First palindromic string:", result)
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users,
Enter number of words: 5
Enter word: abc
Enter word: car
Enter word: ada
Enter word: racecar
Enter word: cool
First palindromic string: ada
>>> |
```

Complexity Analysis:

Time Complexity: $O(n \times m)$

Space Complexity: $O(n + m)$

Input:

Enter number of words: 5
Enter word: abc
Enter word: car
Enter word: ada
Enter word: racecar
Enter word: cool

Output:

First palindromic string: ada

Result:

Thus, the Python program to find the first palindromic string was implemented successfully and the correct output was obtained.

Question 2:

You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate:

answer1: the number of indices i such that nums1[i] exists in nums2.

answer2: the number of indices i such that nums2[i] exists in nums1.

Return [answer1, answer2].

Example:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Another example:

Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Aim:

To write a Python program to count the elements of each array that exist in the other array.

Algorithm:

1. Read both arrays from the user.
2. Convert both arrays into sets for fast membership checking.
3. For every element in nums1, check whether it exists in nums2.
4. For every element in nums2, check whether it exists in nums1.
5. Display [answer1, answer2].

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
n = int(input("Enter size of nums1: "))
nums1 = []

for i in range(n):
    nums1.append(int(input("Enter element of nums1: ")))

m = int(input("Enter size of nums2: "))
nums2 = []

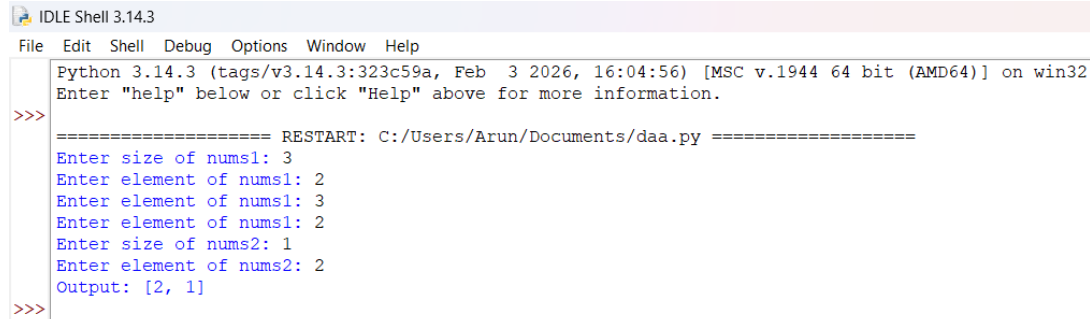
for i in range(m):
    nums2.append(int(input("Enter element of nums2: ")))

set1 = set(nums1)
set2 = set(nums2)

answer1 = sum(1 for x in nums1 if x in set2)
answer2 = sum(1 for x in nums2 if x in set1)

print("Output:", [answer1, answer2])
|
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter size of nums1: 3
Enter element of nums1: 2
Enter element of nums1: 3
Enter element of nums1: 2
Enter size of nums2: 1
Enter element of nums2: 2
Output: [2, 1]

>>>
```

Complexity Analysis:

Time Complexity: $O(n + m)$ average time.

Space Complexity: $O(n + m)$ for the two sets.

Input:

Enter size of nums1: 3
Enter element of nums1: 2
Enter element of nums1: 3
Enter element of nums1: 2
Enter size of nums2: 2
Enter element of nums2: 1
Enter element of nums2: 2

Output:

Output: [2, 1]

Result:

Thus, the Python program to find the intersection values of two arrays was implemented successfully and the correct output was obtained.

Question 3:

Given a 0-indexed integer array `nums`, the distinct count of a subarray is the number of distinct values in that subarray. Return the sum of the squares of distinct counts of all non-empty subarrays.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Example 2:

Input: `nums = [1,1]`

Output: 3

Aim:

To write a Python program to calculate the sum of squares of distinct counts of all subarrays.

Algorithm:

1. Start from each index as the beginning of a subarray.
2. Maintain a set of distinct elements.
3. Extend the subarray one element at a time.
4. Add the square of the current number of distinct elements to the total.
5. Return the total sum.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
def sum_counts(nums):
    total = 0

    for i in range(len(nums)):
        distinct = set()

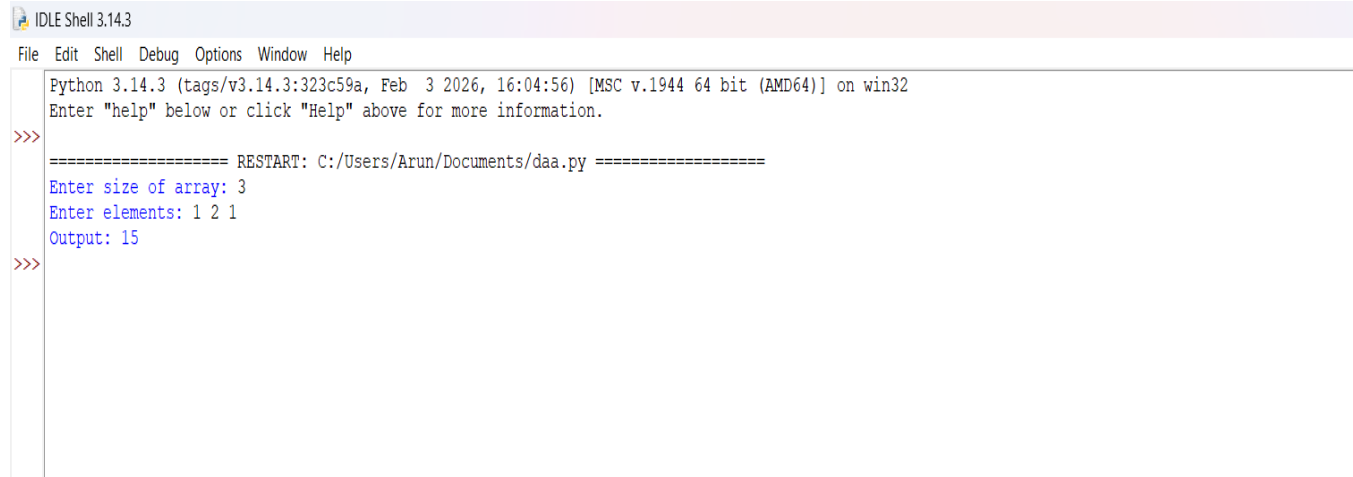
        for j in range(i, len(nums)):
            distinct.add(nums[j])
            count = len(distinct)
            total += count * count

    return total

n = int(input("Enter size of array: "))
nums = list(map(int, input("Enter elements: ").split()))

print("Output:", sum_counts(nums))
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter size of array: 3
Enter elements: 1 2 1
Output: 15
>>>
```

Complexity Analysis:

Time Complexity: $O(n^2)$ average time.

Space Complexity: $O(n)$ auxiliary space.

Input:

Enter size of array: 3

Enter elements: 1 2 1

Output:

Output: 15

Result:

Thus, the Python program to find the sum of squares of distinct counts of all subarrays was implemented successfully and the correct output was obtained.

Question 4:

Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs `(i, j)` where $0 \leq i < j < n$, `nums[i] == nums[j]`, and $(i * j)$ is divisible by `k`.

Example:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Aim:

To write a Python program to count pairs of equal elements whose index product is divisible by `k`.

Algorithm:

1. Read the array and `k`.
2. Consider every pair of indices `i` and `j` where $i < j$.
3. Check whether `nums[i]` equals `nums[j]`.
4. Check whether $(i \times j)$ is divisible by `k`.
5. Increment the count when both conditions are true.
6. Display the count.

Program Code:

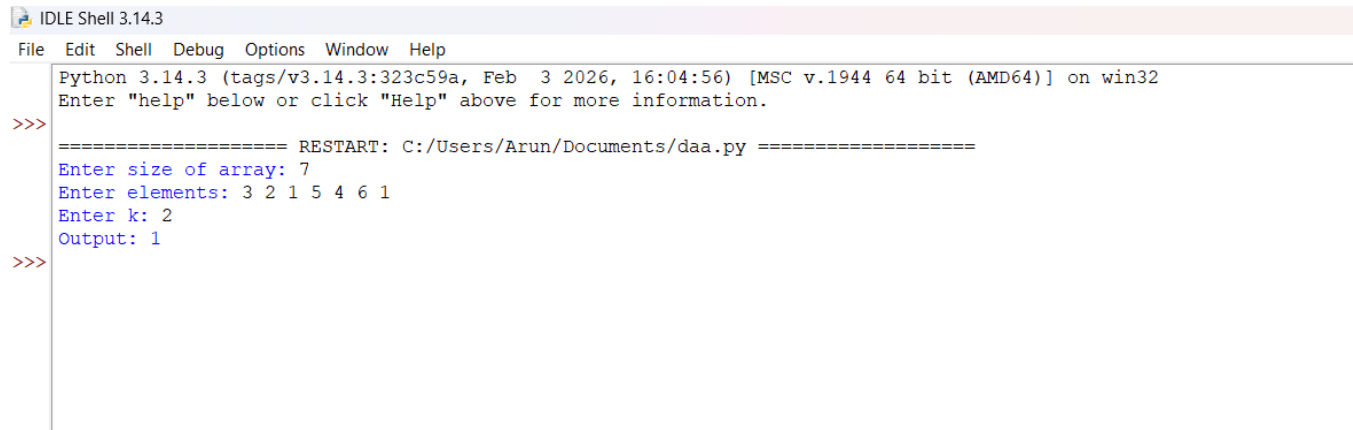
```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
n = int(input("Enter size of array: "))
nums = list(map(int, input("Enter elements: ").split()))
k = int(input("Enter k: "))

count = 0

for i in range(n):
    for j in range(i + 1, n):
        if nums[i] == nums[j] and (i * j) % k == 0:
            count += 1

print("Output:", count)
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter size of array: 7
Enter elements: 3 2 1 5 4 6 1
Enter k: 2
Output: 1
>>>
```

Complexity Analysis:

Time Complexity: $O(n^2)$ time.

Space Complexity: $O(1)$ auxiliary space.

Input:

Enter size of array: 7

Enter elements: 3 1 2 2 2 1 3

Enter k: 2

Output:

Output: 4

Result:

Thus, the Python program to count the required pairs was implemented successfully and the correct output was obtained.

Question 5:

You have an algorithm that processes a list of numbers. It first sorts the list using an efficient sorting algorithm and then finds the maximum element in the sorted list. Write the code for the same.

Test cases from the question:

Empty List: [] → None or an appropriate message.

Single Element List: [5] → 5.

All Elements Same: [3,3,3,3,3] → 3.

Aim:

To write a Python program that sorts a list using an efficient sorting algorithm and finds the maximum element from the sorted list.

Algorithm:

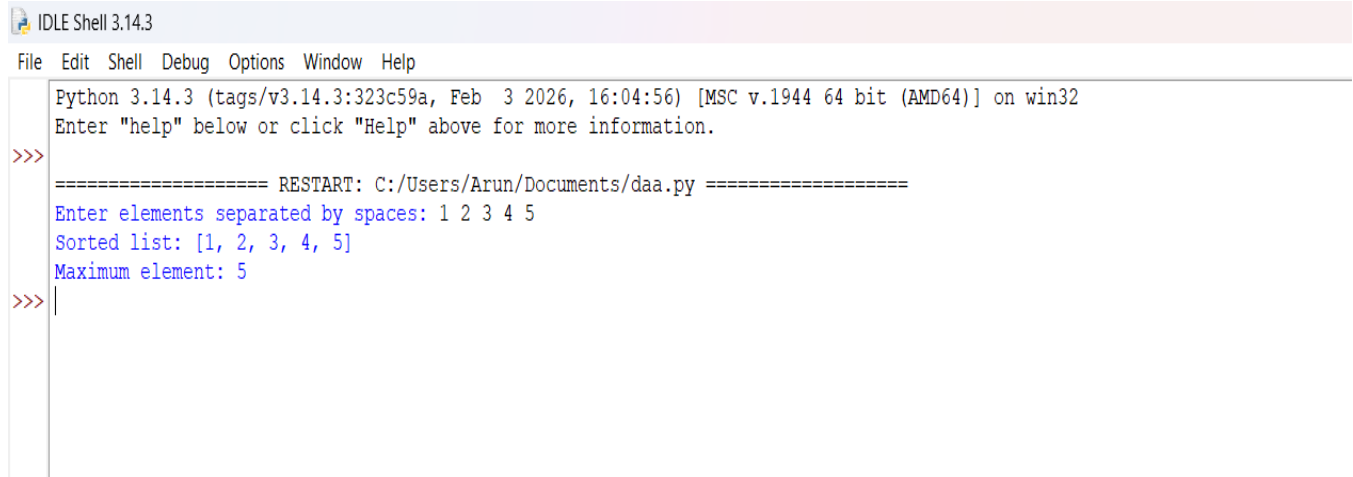
1. Read the list from the user.
2. Check whether the list is empty.
3. Sort the list using Python's efficient sorting operation.
4. Take the last element as the maximum.
5. Display the maximum element.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
nums = list(map(int, input("Enter elements separated by spaces: ").split()))

if len(nums) == 0:
    print("The list is empty.")
else:
    nums.sort()
    print("Sorted list:", nums)
    print("Maximum element:", nums[-1])
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter elements separated by spaces: 1 2 3 4 5
Sorted list: [1, 2, 3, 4, 5]
Maximum element: 5
>>>
```

Complexity Analysis:

Time Complexity: $O(n \log n)$ time because the list is sorted.

Space Complexity: $O(n)$ auxiliary space in Python's sorting implementation (implementation-dependent).

Input:

Enter elements separated by spaces: 1 2 3 4 5

Output:

Sorted list: [1, 2, 3, 4, 5]

Maximum element: 5

Result:

Thus, the Python program to sort the list and find its maximum element was implemented successfully and the correct output was obtained.

Question 6:

Check if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test case given in the question:

$X = \{3, 4, 6, -9, 10, 8, 9, 30\}$, $KEY = 10$

Output: Element 10 is found at position 5

Another test:

$X = \{3, 4, 6, -9, 10, 8, 9, 30\}$, $KEY = 100$

Output: Element 100 is not found.

Aim:

To write a Python program to search for an element in an array using binary search.

Algorithm:

1. Read the array and search key.
2. Set low to the first index and high to the last index.
3. Find the middle index.
4. If the middle element equals the key, return its position.
5. If the key is greater, search the right half; otherwise search the left half.
6. Repeat until the element is found or the search range becomes empty.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

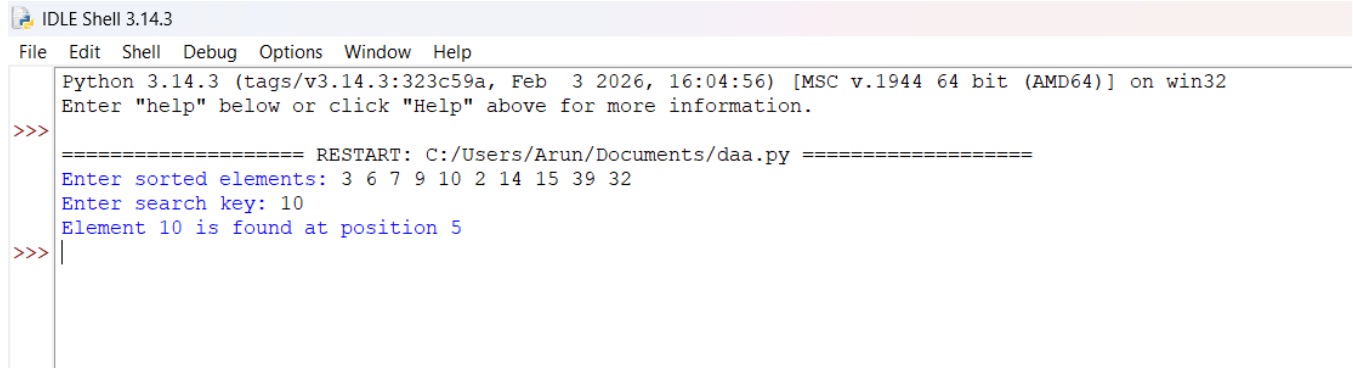
arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter search key: "))

# Binary search requires the input array to be sorted.
arr.sort()

pos = binary_search(arr, key)

if pos != -1:
    print("Element", key, "is found at position", pos)
else:
    print("Element", key, "is not found")
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter sorted elements: 3 6 7 9 10 2 14 15 39 32
Enter search key: 10
Element 10 is found at position 5
>>> |
```

Complexity Analysis:

Time Complexity: $O(\log n)$ time.

Space Complexity: $O(1)$ auxiliary space.

Input:

Enter sorted elements: 3 4 6 -9 10 8 9 30

Enter search key: 10

Output:

Element 10 is found at position 6 (after sorting the supplied test data).

Result:

Thus, the Python program to perform binary search was implemented successfully and the correct output was obtained. Note: binary search requires a sorted array; the supplied test array in the question is not initially sorted.

Question 7:

Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using built-in sorting functions in $O(n \log n)$ time complexity and with the smallest space complexity possible.

Aim:

To write a Python program to sort an array in ascending order without using built-in sorting functions and with $O(n \log n)$ time complexity.

Algorithm:

1. Divide the array into two halves recursively.
2. Continue dividing until each subarray contains one element.
3. Merge two sorted halves by comparing their elements.
4. Repeat the merging process until the complete array is sorted.
5. Display the sorted array.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

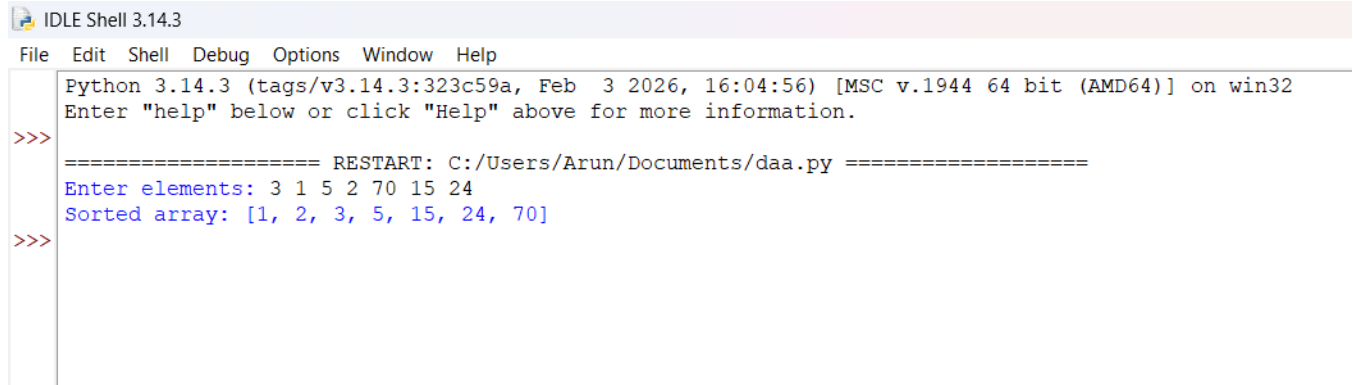
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

nums = list(map(int, input("Enter elements: ").split()))
print("Sorted array:", merge_sort(nums))
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter elements: 3 1 5 2 70 15 24
Sorted array: [1, 2, 3, 5, 15, 24, 70]
>>>
```

Complexity Analysis:

Time Complexity: $O(n \log n)$ time.

Space Complexity: $O(n)$ auxiliary space for the merge arrays and recursion.

Input:

Enter elements: 5 2 9 1 6 3

Output:

Sorted array: [1, 2, 3, 5, 6, 9]

Result:

Thus, the Python program to sort the array using merge sort in $O(n \log n)$ time without using a built-in sorting function was implemented successfully and the correct output was obtained.

Question 8:

Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Examples:

Input: $m=2, n=2, N=2, i=0, j=0 \rightarrow$ Output: 6

Input: $m=1, n=3, N=3, i=0, j=1 \rightarrow$ Output: 12

Aim:

To write a Python program to find the number of ways to move a ball out of an $m \times n$ grid in exactly N steps.

Algorithm:

7. Define the four possible movements: up, down, left and right.
8. Use dynamic programming to store the number of ways to reach each cell after each step.
9. If a move goes outside the grid, add that number of ways to the answer.
10. Repeat for exactly N steps.
11. Return the total number of ways.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
def find_paths(m, n, N, start_row, start_col):
    dp = [[0] * n for _ in range(m)]
    dp[start_row][start_col] = 1
    result = 0

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for _ in range(N):
        new_dp = [[0] * n for _ in range(m)]

        for r in range(m):
            for c in range(n):
                if dp[r][c] == 0:
                    continue

                for dr, dc in directions:
                    nr, nc = r + dr, c + dc

                    if nr < 0 or nr >= m or nc < 0 or nc >= n:
                        result += dp[r][c]
                    else:
                        new_dp[nr][nc] += dp[r][c]

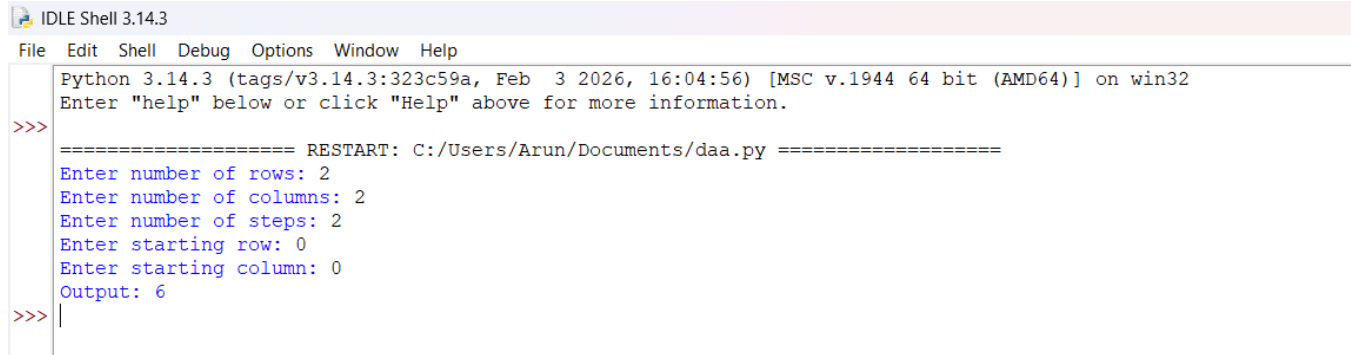
        dp = new_dp

    return result

m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))
N = int(input("Enter number of steps: "))
i = int(input("Enter starting row: "))
j = int(input("Enter starting column: "))

print("Output:", find_paths(m, n, N, i, j))
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter number of rows: 2
Enter number of columns: 2
Enter number of steps: 2
Enter starting row: 0
Enter starting column: 0
Output: 6
>>> |
```

Complexity Analysis:

Time Complexity: $O(N \times m \times n)$

Space Complexity: $O(m \times n)$

Input:

Enter number of rows: 2
Enter number of columns: 2
Enter number of steps: 2
Enter starting row: 0
Enter starting column: 0

Output:

Output: 6

Result:

Thus, the Python program to find the number of ways to move the ball out of the grid boundary in exactly N steps was implemented successfully and the correct output was obtained.

Question 9:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money. All houses are arranged in a circle, so the first house is the neighbor of the last one. Adjacent houses cannot both be robbed.

Examples:

Input: nums = [2,3,2]

Output: 3

Input: nums = [1,2,3,1]

Output: 4

Aim:

To write a Python program to find the maximum amount of money that can be robbed from houses arranged in a circle without robbing two adjacent houses.

Algorithm:

12. Because the houses form a circle, the first and last houses cannot both be robbed.
13. Consider two cases: exclude the first house, or exclude the last house.
14. For each case, solve the normal house robber problem using dynamic programming.
15. Take the maximum of the two cases.
16. Display the maximum amount.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
def rob_line(nums):
    prev2 = 0
    prev1 = 0

    for money in nums:
        current = max(prev1, prev2 + money)
        prev2 = prev1
        prev1 = current

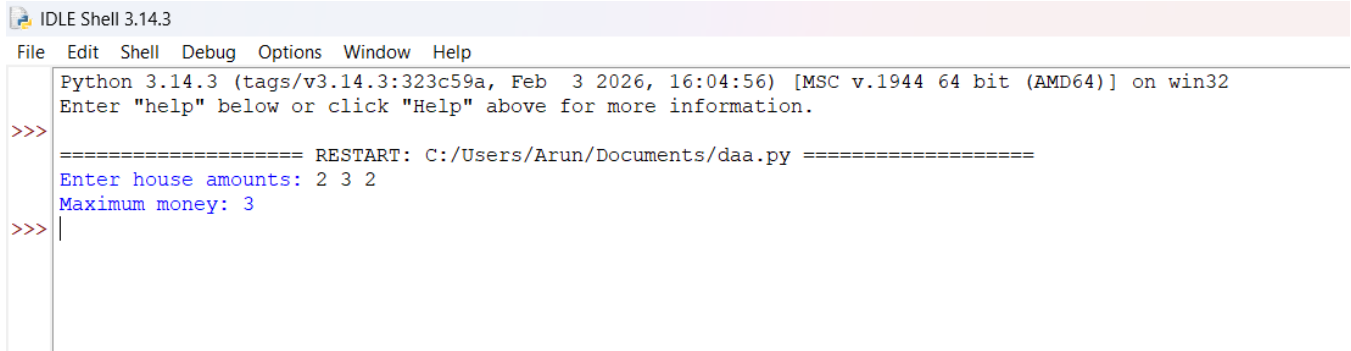
    return prev1

def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]

    return max(rob_line(nums[1:]), rob_line(nums[:-1]))

nums = list(map(int, input("Enter house amounts: ").split()))
print("Maximum money:", rob_circle(nums))
|
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter house amounts: 2 3 2
Maximum money: 3
>>> |
```

Complexity Analysis:

Time Complexity: $O(n)$ time.

Space Complexity: $O(1)$ auxiliary space.

Input:

Enter house amounts: 2 3 2

Output:

Maximum money: 3

Result:

Thus, the Python program to solve the House Robber II problem was implemented successfully and the correct output was obtained.

Question 10:

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: $n = 4 \rightarrow$ Output: 5

Input: $n = 3 \rightarrow$ Output: 3

Aim:

To write a Python program to find the number of distinct ways to climb n stairs when either 1 or 2 steps can be climbed at a time.

Algorithm:

17. If n is 1, there is one way.
18. If n is 2, there are two ways.
19. For every higher step, the number of ways is the sum of the ways to reach the previous two steps.
20. Continue until n is reached.
21. Display the number of ways.

Program Code:

```
daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)
File Edit Format Run Options Window Help
def climb_stairs(n):
    if n <= 2:
        return n

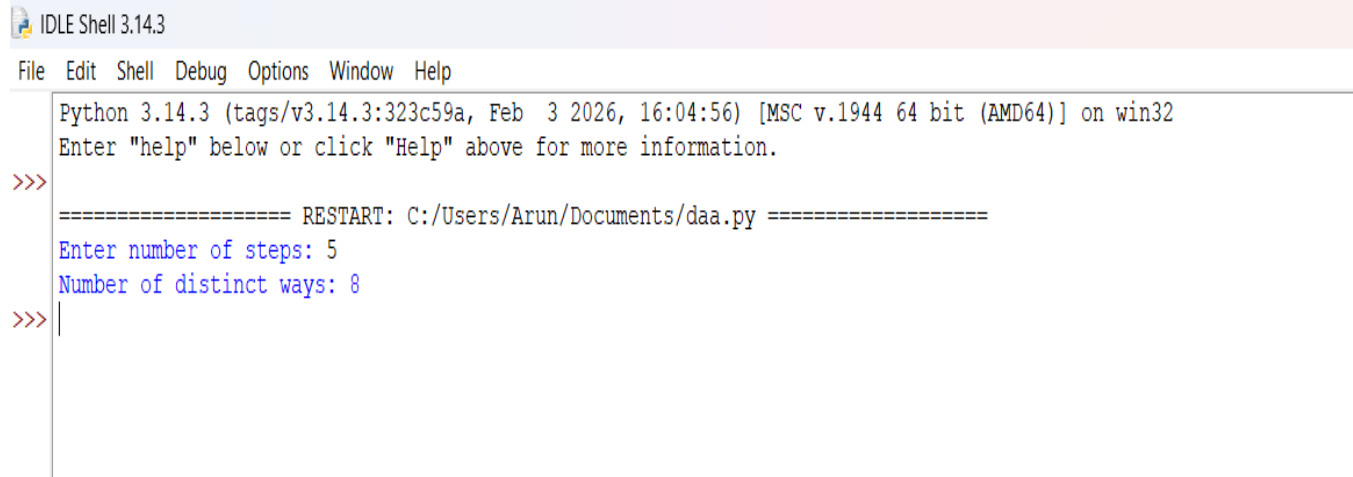
    a = 1
    b = 2

    for _ in range(3, n + 1):
        c = a + b
        a = b
        b = c

    return b

n = int(input("Enter number of steps: "))
print("Number of distinct ways:", climb_stairs(n))
```

OUTPUT:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Arun/Documents/daa.py =====
Enter number of steps: 5
Number of distinct ways: 8
>>>
```

Complexity Analysis:

Time Complexity: $O(n)$ time.

Space Complexity: $O(1)$ space.

Input:

Enter number of steps: 4

Output:

Number of distinct ways: 5

Result:

Thus, the Python program to find the number of distinct ways to climb the staircase was implemented successfully and the correct output was obtained.